

P2986R0

Generic Function Pointer

Published Proposal, 2023-10-14

Author:

[Lauri Vasama](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current draft:

vasama.github.io/wg21/D2986.html

Current draft source:

github.com/vasama/wg21/blob/main/D2986.bs

Abstract

We propose to introduce an equivalent of the void pointer for constexpr type-erasure of function pointers.

Table of Contents

1 Motivation

1.1 `std::function_ref`

2 Existing alternatives

2.1 Conversion to `void*`

2.2 Conversion to `R(*)()`

3 Design

3.1 Example usage in `function_ref`

4 C compatibility

5 Proposed Wording (incomplete)

References

Informative References

§ 1. Motivation

Since C++ was first standardised, `void*` has served to erase the types of pointers to objects. type-erasure of functions, as well as type-erasure in general, is known to be widely useful. This is exemplified by the existence of the standard library templates `std::function`, `std::move_only_function`, and the `std::function_ref` now accepted for C++26, as well the existing language mechanisms for directly type-erasing function pointers (see § 2 Existing alternatives).

None of the currently available mechanisms for type-erasure of function pointers are available during constant evaluation however. With [P2738R1] now accepted for C++26, and bringing with it `constexpr` conversion from `void*` back to the original type, these existing solutions seem especially lacking in comparison.

§ 1.1. `std::function_ref`

[P0792R14] introducing `function_ref`, a type-erased callable reference, has been accepted for C++26. `function_ref` notably lacks `constexpr` support for invocation and for construction from a function pointer:

```
// [func.wrap.ref.ctor], constructors and assignment operators
template function_ref(F*) noexcept;

// [func.wrap.ref.inv], invocation
R operator()(ArgTypes...) const noexcept(noex);
```

Neither of these operations is implementable in `constexpr` due to the lack of a `constexpr` type-erasure mechanism for function pointers. From [func.wrap.ref.class]:

An object of class `function_ref<R(Args...) cv noexcept(noex)>` stores a pointer to function *thunk_ptr* and an object *bound-entity*. *bound-entity* has an unspecified trivially copyable type `BoundEntityType`, that models copyable and is capable of storing a pointer to object value or a pointer to function value. The type of *thunk_ptr* is `R(*) (BoundEntityType, Args&&...) noexcept(noex)`.

C++ currently offers no way to implement the described `BoundEntityType` for use during constant evaluation because it must be “capable of storing ... a pointer to function value.”

The reference implementation for `function_ref` can be found at [zhihaoy/nontype_functional@v1.0.1](https://github.com/zhihaoy/nontype_functional@v1.0.1).

Here is the definition of `BoundEntityType (storage)` from the reference implementation:

```
struct _function_ref_base
{
    union storage
    {
        void *p_ = nullptr;
        void const *cp_;
        void (*fp_)();

        constexpr storage() noexcept = default;

        template<class T> requires std::is_object_v<T>
        constexpr explicit storage(T *p) noexcept : p_(p)
        {}

        template<class T> requires std::is_object_v<T>
        constexpr explicit storage(T const *p) noexcept : cp_(p)
        {}

        template<class T> requires std::is_function_v<T>
        constexpr explicit storage(T *p) noexcept
            : fp_(reinterpret_cast<decltype(fp_)>(p))
        {}
    };

    template<class T> constexpr static auto get(storage obj)
    {
        if constexpr (std::is_const_v<T>)
            return static_cast<T *>(obj.cp_);
```

```

        else if constexpr (std::is_object_v<T>)
            return static_cast<T *>(obj.p_);
        else
            return reinterpret_cast<T *>(obj.fp_);
    }
};

```

Note that while the function pointer constructor and the function `get` are both marked `constexpr`, due to the use of `reinterpret_cast`, the former can never result in a constant expression and the latter can only do so when `T` is not a function type.

§ 2. Existing alternatives

C++ offers two existing solutions for type-erasure of function pointers, each with its own downsides:

§ 2.1. Conversion to `void*`

Conversions between pointers to functions and pointers to objects are conditionally supported:

```

void print_int(int value) {
    printf("%d\n", value);
}

int main() {
    auto fp = reinterpret_cast<void*>(print_int);
    reinterpret_cast<void(*) (int)>(fp)(42);
}

```

The first downside is obvious: This solution is not portable. Requiring unconditional support for this conversion may exclude some exotic platforms where pointers to functions and pointers to objects have different storage requirements. It may also introduce difficulties for implementation of pointer comparisons on Harvard architectures, where the same bit patterns may be reused for both pointers pointing to data and code.

Moreover, the use of `void*` for type-erasing function pointers conflates data and functions and may lead to accidentally passing the `void*` to functions such as `memcpy` or `free`. Due to the fundamental incompatibility of these pointers, any such code should be ill-formed to prevent mistakes, which is not possible if `void*` is used.

Finally, making the conversion from a function pointer to `void*` implicit may not be backwards compatible due to its effects on overload resolution and it would further exacerbate the type safety issue due to the existing conversions between functions and function pointers.

For these reason we do not consider this to be a viable solution.

§ 2.2. Conversion to `R(*)()`

Conversions between different types of pointers to functions are allowed and converting back to the original type yields the original value:

```
void print_int(int value) {
    printf("%d\n", value);
}

int main() {
    using fp_t = void(*)();
    auto fp = reinterpret_cast<fp_t>(print_int);
    reinterpret_cast<void(*) (int)>(fp)(42);
}
```

This conversion requires the use of a reinterpret cast, which is not a constant expression in either direction. Making reinterpret cast expressions constant expressions would arguably represent a larger change to the language than what is proposed by this paper.

The user must make a choice of some concrete function pointer type to represent a type-erased function pointer, `void(*)()` being an obvious choice. If care is not taken, such converted pointers will remain invocable, which would lead to undefined behaviour. A better but less obvious choice is `void(*) (incomplete)`, where `incomplete` is an incomplete class type, making invocations of such a function pointer type ill-formed. In any case, each user will pick a different type to represent type-erased function pointers.

This solution could be improved upon by making the reinterpret cast a constant expression and introducing an uninvocable function pointer type alias in the standard library for the purpose of type-erasure. However, we believe that even so this solution would be inferior to what is proposed by this paper.

§ 3. Design

Introduce a new core language type, the *generic function pointer type* under the library name `std::function_ptr_t`. This type is a function pointer type behaving similarly to the void pointer:

- `nullptr_t` is implicitly convertible to a generic function pointer. This conversion is a constant expression

```
constexpr std::function_ptr_t fp_0 = nullptr;
```

- Any non-generic function pointer is implicitly convertible to a generic function pointer. This conversion is a constant expression.

```
constexpr int f() {  
    return 42;  
}  
constexpr std::function_ptr_t fp_f = f;
```

- Two generic function pointers can be compared for equality. This comparison is a constant expression.

```
static_assert( fp_0 == nullptr );  
static_assert( fp_f != nullptr );  
  
constexpr int g() {  
    return 0;  
}  
static_assert( fp_f == f );  
static_assert( fp_f != g );
```

- A generic function pointer `fp` is explicitly convertible to a non-generic function pointer type `F*` using `static_cast`. This conversion is a constant expression if the type of the function pointed to by `fp` is exactly `F`.

```
constexpr auto p_f = static_cast<int(*)>(fp_f);  
static_assert( p_f == f );  
static_assert( p_f() == 42 );
```

- Unlike a non-generic function pointer, a generic function pointer can neither be invoked nor

dereferenced.

```
fp_f(); // ill-formed
*fp_f; // ill-formed
```

The alias `std::function_ptr_t` for this type is introduced in the `<cstddef>` header.

§ 3.1. Example usage in `function_ref`

Here is the `function_ref` reference implementation seen in § 1.1 `std::function_ref` again, shown with changes permitted by this proposal to make it fully usable during constant evaluation:

```
struct _function_ref_base
{
    union storage
    {
        void *p_ = nullptr;
        void const *cp_;
        void (*fp_)();
        std::function_ptr_t fp_;

        constexpr storage() noexcept = default;

        template<class T> requires std::is_object_v<T>
        constexpr explicit storage(T *p) noexcept : p_(p)
        {}

        template<class T> requires std::is_object_v<T>
        constexpr explicit storage(T const *p) noexcept : cp_(p)
        {}

        template<class T> requires std::is_function_v<T>
        constexpr explicit storage(T *p) noexcept
            : fp_(reinterpret_cast<decltype(fp_)>(p))
            : fp_(p)
        {}
    };

    template<class T> constexpr static auto get(storage obj)
    {
```

```

        if constexpr (std::is_const_v<T>)
            return static_cast<T *>(obj.cp_);
        else if constexpr (std::is_object_v<T>)
            return static_cast<T *>(obj.p_);
        else
            return reinterpret_cast<T *>(obj.fp_);
            return static_cast<T *>(obj.fp_);
    }
};

```

§ 4. C compatibility

[WG14 N2230] Proposed a similar type under the name `funcptr_t`, and while WG14 expressed interest in such a type, the design presented in that proposal did not gain consensus. The author has not since followed up on that paper.

We intend to propose the introduction of a C++ compatible `function_ptr_t` type to WG14.

§ 5. Proposed Wording (incomplete)

Add a new clause to [\[basic.fundamental\]](#):

The type named by `std::function_ptr_t` is called the *generic function pointer type*. A value of that type can be used to point to functions of unknown type. Such a pointer shall be able to hold any function pointer.

Add a new clause to [\[conv.fctptr\]](#):

A prvalue of type "pointer to function" can be converted to a prvalue of type `std::function_ptr_t`. The pointer value is unchanged by this conversion.

Add a new clause to [\[expr.static.cast\]](#):

A prvalue of type `std::function_ptr_t` can be converted to a prvalue of type "pointer to function". The pointer value is unchanged by this conversion.

Add a new type alias to [\[cstddef.syn\]](#):

```
namespace std {  
    using function_ptr_t = generic function pointer type; // freestanding  
}
```

§ References

§ Informative References

[P0792R14]

Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. *[function_ref: a non-owning reference to a Callable](#)*. 9 February 2023. URL: <https://wg21.link/p0792r14>

[P2738R1]

Corentin Jabot, David Ledger. *[constexpr cast from void*: towards constexpr type-erasure](#)*. 13 February 2023. URL: <https://wg21.link/p2738r1>